LECTURES

07 Window methods | 07 Timers, eval, scrolling | 08 DOM concepts | 09 DOM in practice | ★ Cheat sheet

REVISION NOTES · JAVASCRIPT

JavaScript Window and DOM Notes

Condensed from three decks: Window method, DOM and DOM Example. Code is cleaned up from the slides (curly quotes and typos fixed). Points marked "Extra" go slightly beyond the slides.

Window methods

Deck 07 · Window_method

`window` is the top-level browser object. Its methods can be called without the prefix: `alert()` means `window.alert()`. The deck covers: `open`, `close`, `alert`, `prompt`, `confirm`, `isNaN`, `isFinite`, `eval`, `setInterval`, `clearInterval`, `setTimeout`, `clearTimeout`, and `location` (a getter/setter property).

window.open(URL, name, specs)

```
function callme(){ window.open('p.html'); }
function tool(){ window.open('p.html', '_blank', 'height=200,width=200'); }
```

PARAMETER	MEANING
URL	Optional. Page to open. If left out, a blank window (`about:blank`) opens.
name	Optional. `_blank` opens in a new window (the default), `_self` replaces the current page, or a window name.
specs	Optional. Comma-separated list of window features.

SPEC	EFFECT
`width`, `height`	Size in pixels. Minimum 100.
`left`, `top`	Position in pixels.

`menubar` , `toolbar` , `status` , `scrollbars` , `resizable`	`yes|no|1|0` . Default is yes.
`fullscreen`	Full-screen mode. Default is no.
`titlebar`	Ignored unless the caller is an HTML Application or trusted dialog.

Note: one slide says `open` uses the current window by default, while the parameter table says `_blank` (new window) is the default. Follow the table.

prompt(text, default)

Shows an input box and returns what the user typed as a **string**.

```
var no1 = window.prompt("enter number", 5);   // 5 is the default text
var no2 = prompt();

no1 + no2;            // "56"  string + string: joins
no1 * no2;            // 30    * forces conversion to numbers
typeof no1;           // "string"

parseInt(no1) + parseInt(no2);    // 11
+no1 + +no2;                      // 11  unary + converts too
```

INPUT	PARSEINT	UNARY +
"5" and "6"	5 and 6, sum 11	5 and 6, sum 11
"5.5" and "6.7"	5 and 6, sum 11 (decimals cut off)	5.5 and 6.7, sum 12.2

Use `parseInt` for whole numbers, unary `+` (or `Number()`) when decimals matter. Deck tasks: accept two numbers and show the sum in an `<h1>` ; accept two numbers and show the product.

alert

- Shows plain text. HTML tags are **not** interpreted: `alert("hello <br/> world")` shows the tag as text.
- Use `\n` for a new line: `alert("hello \n world")` .
- `alert` is just a property of `window` , so it can be replaced:

```
alert = function(a){ document.write(a); };
alert("hello");     // now writes into the page instead of showing a dialog
```

confirm

```
a = confirm("do you want to continue");
alert(a);       // OK -> true, Cancel -> false
```

location: redirect the user

```
function newLocation(){ window.location = "p.html"; }    // assigning navigates there
document.getElementById("btn").addEventListener("click", newLocation, false);
```

`location` works as a getter (reads the current URL) and a setter (assigning sends the browser to a new page). The third argument `false` to `addEventListener` means the bubbling phase (the default).

resizeBy, resizeTo, moveBy, moveTo

```
function resizeWindow(){ window.resizeBy(50, 50); }
window.moveBy(50, 50);
```

- `resizeTo` / `moveTo` set an **absolute** size or position, so repeating them changes nothing after the first time.
- `resizeBy` / `moveBy` are **relative** to the current value, so each call adds 50 more.
- The slide says these work only in IE. Extra: modern browsers allow them only on windows opened with `window.open`.

Timers, eval and scrolling

`Deck 07 · Window_method, deck 09 · DOM_Example`

setTimeout and setInterval

	SETTIMEOUT(FN, MS)	SETINTERVAL(FN, MS)
Runs	**Once**, after the delay	**Repeatedly**, every delay
Stop it	`clearTimeout(t)`	`clearInterval(t)`
Returns	A timer id (`t`)	A timer id (`t`)

```
t = setInterval("alert('hi')", 1000);    // every 1000 ms (1 second)
// clearInterval(t)
t = setTimeout("alert('hi')", 6000);     // once, after 6 seconds
// clearTimeout(t)
```

Counter with setTimeout calling itself (start and stop buttons):

```
var c = 0, t;
function timedCount(){
  document.getElementById('txt').value = c;
  c = c + 1;
  t = setTimeout("timedCount()", 1000);   // schedule the next tick
}
function stopCount(){ clearTimeout(t); }
// <input type="button" value="Start count!" onClick="timedCount()">
// <input type="text" id="txt">
// <input type="button" value="Stop count!" onClick="stopCount()">
```

Better way: pass a function

```
function sayHi(){ alert('Hello'); }
setTimeout(sayHi, 1000);                    // function name, no quotes, no ()
setTimeout(() => alert('Hello'), 1000);     // arrow function

document.write("non blocking programing");
document.write("bye");                      // these run first; the timer fires later
```

The timer does not pause the program: `setTimeout` only delays that one job, and the lines after it run immediately. This is non-blocking programming.

WATCH OUT: SETINTERVAL INSIDE THE REPEATED FUNCTION

If the function that `setInterval` calls starts another `setInterval`, every tick adds a new timer. The count speeds up and the old timer id is lost, so `clearInterval(t)` stops only the last one.

```
// Correct: start the interval once, in a separate start function
function st(){ a = 1; t = setInterval("call()", 1000); }
function call(){ document.getElementById("nm").value = a++; }
function tp(){ clearInterval(t); }

// Wrong: call() creates a new interval each time it runs
function call(){ document.getElementById("nm").value = a++; t = setInterval("call
```

eval(): evaluate an expression held in a string

```
a = "5+2-3";
eval(a);                  // 4
eval("5+2-3*6/2");        // -2   (* and / first: 5+2-9 = -2)
eval("55+6");             // 61

for (i = 1; i <= 5; i++) document.write(eval("no" + i));   // reads variables no1..no5
```

- Normal operator precedence applies: `3*6/2` is worked out before the `+` and `-`.
- The last example builds a variable name as text and reads its value. It works with the global `var no1 ... no5` from the prompts.
- Warning noted on the slide: **CSRF / XSS**. `eval` runs any text as code, so passing it user input is a security risk. Avoid it where you can.

Also in the list: `isNaN(x)` tells whether a value is not a number; `isFinite(x)` is true only for a real finite number (false for `Infinity` and `NaN`) (Extra).

Scrolling the window

CALL	EFFECT
`window.scrollBy(x, y)`	Scroll **relative** to where you are. `scrollBy(0, 10)` goes 10px down. Negative values allowed.
`window.scrollTo(x, y)`	Scroll to **absolute** page coordinates. `scrollTo(0, 0)` goes to the top.
`elem.scrollIntoView(top)`	`true` (default): element's top edge at the top of the window. `false`: bottom edge at the bottom.
`window.onscroll`	Event that fires while scrolling.

```
<button onclick="window.scrollBy(0,10)">scrollBy(0,10)</button>
<button onclick="window.scrollTo(0,0)">scrollTo(0,0)</button>
<button onclick="this.scrollIntoView()">scrollIntoView()</button>
<button onclick="this.scrollIntoView(false)">scrollIntoView(false)</button>
```

- Scrolling needs the DOM to be fully built. A scroll call in a script in the `<head>` will not work.
- Regular elements scroll by changing `scrollTop` / `scrollLeft`; for the page use `document.documentElement.scrollTop` (Safari: `document.body.scrollTop`), or simply use the methods above, which work the same in all browsers.

simply use the methods above, which work the same in all browsers.

Forbid scrolling

```
document.body.style.overflow = "hidden";   // page freezes at the current position
document.body.style.overflow = "";         // released again
```

Used when a message must get the visitor's attention. Drawback: the scrollbar disappears and content jumps to fill the space. Fix: compare `clientWidth` before and after, and add that difference as padding to the body. The same trick works on other elements.

DOM concepts

Deck 08 · DOM

DOM stands for **Document Object Model**. The browser turns the HTML page into a tree of objects that JavaScript can read and change.

Node types

PART OF THE PAGE	NODE	NODETYPE	NODENAME
Whole document	Document node	9	`#document`
Each HTML element	Element node	1	The tag name, e.g. `P`
Each attribute	Attribute node	2	The attribute name
Text inside elements	Text node	3	`#text`
Comments	Comment node	8	`#comment`

`nodeName` is read-only.

The tree

```
<html>
  <head><title>DOM Tutorial</title></head>
  <body>
    <h1>DOM Lesson one</h1>
    <p>Hello <span>world</span></p>
  </body>
</html>
```

```
html
├─ head
│   └─ title
│       └─ #text "DOM Tutorial"
└─ body
    ├─ h1  └─ #text "DOM Lesson one"
    └─ p
        ├─ #text "Hello "
        └─ span  └─ #text "world"
```

- `html` is the parent of `head` and `body`; they are its children and each other's siblings.
- Parent, child and sibling relationships are what `parentNode` and `childNodes` follow.

TEXT IS ALWAYS STORED IN TEXT NODES

A common mistake is to expect an element node to hold its text. In `<title>DOM Tutorial</title>`, the element node `title` holds a

TEXT NODE

whose value is "DOM Tutorial". The text is not the value of the `title` element itself. In the HTML DOM you can reach the text with `innerHTML`.

Properties

PROPERTY	GIVES
`x.innerHTML`	The content (text and tags) inside x
`x.nodeName`	The name of x
`x.nodeValue`	The value of x (for a text node, its text)
`x.parentNode`	The parent node
`x.childNodes`	The child nodes, as a list
`x.attributes`	The attribute nodes

Methods

METHOD	DOES
`x.getElementById(id)`	Gets the element with that id
`x.getElementsByTagName(name)`	Gets all elements with that tag name
`x.appendChild(node)`	Inserts a child node into x

`x.appendChild(node)`	Inserts a child node into x
`x.removeChild(node)`	Removes a child node from x

In both lists, `x` is a node object (an HTML element).

DOM in practice

`Deck 09 · DOM_Example`

getElementById and the child nodes

```
<p onmouseOver="callme()" id="pr">Welcome<b> to world of</b> Dom</p>

function callme(){
  let s = document.getElementById("pr");
  alert(s);                          // [object HTMLParagraphElement]
  alert(s.nodeName);                 // P
  alert(s.childNodes[0].nodeValue);  // "Welcome"   first child: text of the first par
  alert(s.childNodes[0].nodeName);   // #text
  alert(s.childNodes[0].nodeType);   // 3
  alert(s.innerHTML);                // everything between the p tags, tags included
}
```

INDEX	CHILD OF `P`	VALUE
`childNodes[0]`	#text	"Welcome"
`childNodes[1]`	`<b>` element, which has its own child #text	`childNodes[1].childNodes[0].nodeValue` is " to world of"
`childNodes[2]`	#text	" Dom"

`getElementById` returns **one object** for the element. `innerHTML` returns the whole inner markup, while `nodeValue` of a text node returns just that piece of text.

Copy from one element to another

```
function call(){
  var x = document.getElementById("cp");
  var y = document.getElementById("pr");
  y.innerHTML = x.innerHTML;       // read from one, write to the other
}
```

Reading a text box uses `.value` instead: `b.value = a.value`. For a paragraph or div, use `innerHTML` (or the first child's `nodeValue`).

getElementsByTagName: a list of elements

```
function callme(){
  let s = document.getElementsByTagName("p");   // collection of all p tags
  let l = s.length;
  for (i = 0; i < l; i++) alert(s[i].innerHTML);
}
```

BE CAREFUL: WHERE THE SCRIPT SITS

Code runs top to bottom. If the script is placed

ABOVE

the `<p>` tags, those elements do not exist yet, so `getElementsByTagName("p")` returns an empty list and nothing is printed. Put the script after the elements, or run it from an event or `onload`.

getElementsByClassName

```
<ul class="example"><li class="child">Coffee</li><li class="child">Tea</li></ul>
<ul class="example"><li class="child">bun</li><li class="child">ice-cream</li></ul>

var list = document.getElementsByClassName("example");   // 2 elements
list[0].getElementsByClassName("child")[0].innerHTML = "Milk";   // Coffee becomes Mil
```

The result is a list (index from 0). You can call `getElementsBy...` on an element to search only inside it.

querySelector and querySelectorAll

METHOD	RETURNS
`querySelector("p")`	The **first** match only
`querySelectorAll("p")`	**All** matches, as a list

```
var x = document.getElementById("myDIV").querySelectorAll("p");
x[0].style.backgroundColor = "red";
x[1].style.backgroundColor = "red";

document.getElementById("myDIV").querySelector("p").style.color = "blue";
```

```
document.querySelector('#foobar');
document.querySelector(".myclass");
document.querySelector("div.user-panel.main input[name='login']");
```

They take any CSS selector:

SELECTOR	MEANING
`#id` , `.class` , `tag`	By id, class or tag
`h1 span` (space)	Any `span` inside an `h1`
`h1 > span`	Direct child only
`h1 + p`	The `p` immediately after an `h1`
`h1 ~ p`	Any `p` sibling after an `h1`
`*`	Everything
`[name='login']`	By attribute value. Also `^=` starts with, `$=` ends with, `*=` contains

getAttribute and setAttribute

```
<p align="center" id="pp">hello</p>
document.getElementById("pp").setAttribute("align", "left");        // write
document.getElementById("dd").innerHTML =
  document.getElementById("pp").getAttribute("align");              // read: "left"
```

Changing style dynamically

```
function callme(){
  document.getElementById("pr").style.color = "red";
  document.getElementById("pr").style.background = "yellow";
}
function change(){
  document.getElementById("pr").style.color = "black";
  document.getElementById("pr").style.background = "pink";
}
// <p id="pr" onmouseover="callme()" onmouseout="change()">...</p>
```

Style properties use camelCase: CSS `background-color` becomes `style.backgroundColor` .

Setting a whole class: className

```
ob = document.getElementById("pr");
ob.className = "cl";       // replaces ALL classes with "cl"
```

classList: work with individual classes

```
var x = document.getElementById("myDIV").classList;   // list of the element's classes
x.length;     // 3
x[0];         // "mystyle"
```

CALL	DOES	RETURNS
`classList.add("anotherClass")`	Adds a class	nothing
`classList.remove("anotherClass")`	Removes a class	nothing
`classList.replace("anotherClass", "myClass")`	Swaps one class for another	Boolean (Extra)
`classList.contains("anotherClass")`	Checks for a class	`true` / `false`
`classList.toggle("anotherClass")`	Removes the class if present, adds it if missing	`true` if the class is now on, `false` if now off

```
// Better: look up the element once and reuse it
let divobj = document.getElementById("myDIV");
y = divobj.classList;
y.add("anotherClass");
document.getElementById("demo").innerHTML = y[2];

// toggle works like this internally
function toggleClass(){
  var x = document.getElementById("myDIV").classList.contains("anotherClass");
  if (x == true){ document.getElementById("myDIV").classList.remove("anotherClass"); r
  else            { document.getElementById("myDIV").classList.add("anotherClass");
}
```

CLASSNAME	CLASSLIST
One string holding all classes. Assigning replaces them all	A list with methods. You change one class and keep the rest

Note: `classList` is not supported in Internet Explorer 9 and earlier.

document.images

```
// <img src="a.bmp" name="mg" id="im" onmouseover="callme()" onmouseout="backimg()">
function callme(){
  alert(document.images.length);              // number of images on the page
  document.mg.src = "b.bmp";                  // by the name attribute
  // document.images[0].src = "b.bmp";        // by index
  document.getElementById("im").src = "b.bmp"; // by id
}
function backimg(){ document.mg.src = "a.bmp"; }
```

All images on the page are stored in the `document.images` array in page order (here `a.bmp`, `p.bmp`, `x.bmp`). Changing `src` swaps the picture, which gives a rollover effect.

Practice tasks from the deck

1. A div with your name; on mouse over, copy it into an `<h1>`.
2. For `<p id="pp">hello<b>bye</b></p>`, read "bye" using the DOM and show it in a div.
3. Read 5 `<p>` tags and show them in a div.
4. Two textareas and a checkbox; clicking the checkbox copies the address from one to the other.
5. Two text boxes and a button that shows their sum.
6. A styled div with 5 lines; on mouse over copy its data into a `<p>`.
7. A resume in HTML.
8. Password and re-enter password; if they differ, show a red error in a span.
9. Three radio buttons (Red, Green, Blue) that change a paragraph's text colour.

```
// Password match check (task 8)
function check(){
  var p1 = document.getElementById("p1").value;
  var p2 = document.getElementById("p2").value;
  var msg = document.getElementById("err");
  if (p1 != p2){ msg.style.color = "red"; msg.innerHTML = "Passwords do not match"; }
  else msg.innerHTML = "";
}

// Radio colour (task 9): <input type="radio" name="c" value="red" onclick="paint(this
function paint(colour){ document.getElementById("para").style.color = colour; }
```

Last-minute cheat sheet

`All three decks`

Likely exam questions

- `prompt` , `confirm` , `alert` : what each returns; why `prompt` values need conversion.
- `window.open` parameters and common specs; how `location` redirects.
- `setTimeout` vs `setInterval` ; how to stop each; why a repeated function must not start a new interval.
- What `eval` does and why it is risky.
- `scrollBy` vs `scrollTo` vs `scrollIntoView` ; how to freeze scrolling.
- What the DOM is; the five node types and their `nodeType` numbers (1, 2, 3, 8, 9).
- Why text is stored in a text node; `innerHTML` vs `nodeValue` .
- `getElementById` vs `getElementsByTagName` vs `getElementsByClassName` vs `querySelector(All)` .
- The `classList` methods; `className` vs `classList` .
- Change style, attribute and image `src` from JavaScript.

Easy mistakes

- Adding two `prompt` values without converting: "5" + "6" is "56".
- Using `parseInt` on decimals and losing the fraction.
- Putting a DOM script above the elements it reads, so the list is empty.
- Treating `getElementsBy...` results as one element. They are lists, use an index.
- Writing `style.background-color` . Use `style.backgroundColor` .
- Assigning `className` and wiping out the other classes.
- Passing `setTimeout` a function call with brackets: `setTimeout(sayHi(), 1000)` runs it at once. Write `sayHi` .
- Expecting `alert("a <br> b")` to break the line. Use `\n` .
- Feeding user input into `eval` .